

CS2D: Manual de Proyecto

Facultad de Ingeniería de la Universidad de
Buenos Aires

Taller de Programación

Cátedra Veiga



Integrantes:

Alumno	Padrón
Fernandez, Facundo	109097
Molina , Taiel ALEXIS	109458
Rebollo, Matías Gabriel	109671
Rocha Diaz, Tomás	111085

Corrector: Capón, Mateo

Contenido

Introducción	2
Desarrollo	2
División de tareas	2
Definiciones iniciales y comienzo de los sprints	2
Herramientas utilizadas por el equipo	3
Documentación consultada	3
Oportunidades de mejora para la materia y el curso	3
Autocrítica grupal	4
Reflexión final para futuros alumnos de Taller	5

Introducción

El presente informe busca informar cómo fue la división del trabajo práctico de Taller de Programación I cátedra Veiga del primer cuatrimestre del año 2025.

A mediados de cuatrimestre se nos indicó que debíamos desarrollar un Counter-Strike 2D en el lenguaje C++, el cual incluía tres programas:

- un programa cliente, que sería utilizado por los jugadores, constando de una interfaz gráfica dividida primero en la librería QT para realizar la etapa de lobby (donde los jugadores se registran, eligen sus skins y crean o se unen a una partida); y luego la etapa del juego en sí sería desarrollado en la librería SDL.
- un programa servidor, que manejaría los múltiples clientes y sus partidas, y la lógica del juego en sí.
- un programa editor, en el cual los usuarios pueden crear mapas para después jugar en ellos, desarrollado al igual que la etapa de lobby con la librería QT.

Desarrollo

División de tareas

Por recomendación de nuestro corrector designado Mateo, decidimos dividirnos las tareas de la siguiente forma:

- Lógica de juego: Fernandez, Facundo.
- Arquitectura del servidor y protocolo: Molina, Taiel.
- Interfaz gráfica del juego (SDL): Rebollo, Matías.
- Lobby y editor de niveles: Rocha Díaz, Tomás.

Definiciones iniciales y comienzo de los sprints

Si bien al inicio nos costó un poco agarrar el ritmo de trabajo, a medida que fueron pasadas las semanas y que fuimos superando los primeros obstáculos (y como siempre en el desarrollo de software, suelen ser los más difíciles) el producto comenzó a escalar rápidamente, elevando la velocidad de trabajo semana a semana.

Las primeras dos semanas hicimos un planteo inicial de lo que serían las distintas partes del producto, exceptuando el editor que quedó para más tarde. Luego de las primeras dos semanas ya teníamos definido el clock propio del juego (parte importante para empezar a trabajar), las snapshots que enviaría el servidor, los pedidos que podría hacer un cliente y el protocolo. Sumado a esto,

teníamos nuestra primera maqueta de lo que sería el mapa, una representación simple en SDL de como sería el juego y el lobby a groso modo terminado.

Durante lo que restó de desarrollo, fuimos iterando sobre lo ya mencionado, mejorando principalmente la interfaz gráfica del lobby, definiendo casos bordes sobre aspectos del juego y demás.

Finalmente la última semana fue cuando pudimos agregar las animaciones, desde los disparos, hasta la explosión de la bomba, y por primera vez pudiendo iterar el proyecto con su tamaño casi completo. Durante los últimos días iteramos bastante los casos bordes y refinando cuestiones de jugabilidad para mejorarle la UX a los futuros jugadores.

Herramientas utilizadas por el equipo

Todos en el grupo utilizamos las mismas herramientas para el desarrollo puro que fueron los linters que recomendó la cátedra al inicio de cursada (clang-format, cppcheck y cpplint) y Visual Studio Code como IDE. Creemos fuertemente que fueron de mucha ayuda instalar los hooks en el repositorio para así no subir código que contenía errores, y también, exceptuando aislados casos, siempre subir código que compilaba. Las únicas veces que no lo hicimos fue para que otra persona del grupo pudiera agregar algo de su parte para que en conjunto funcionen.

Documentación consultada

Por obvias razones, todos hemos consultado la documentación oficial de C++. También hemos utilizado la documentación brindada por Martín Di Paola, docente y Jefe de Trabajos Prácticos de la materia, el cual tiene un blog, en el cual explica en distintas publicaciones, diversas cuestiones de utilidad para el desarrollo de videojuegos en general, y en específico, para nuestro Counter-Strike.

Además de esto, se ha consultado las documentaciones de SDL y la de QT, siendo esta última fuertemente recomendada, porque si bien hay soluciones que la librería en sí no brinda, tiene publicaciones en su foro donde explican al detalle como realizar ciertos features con la misma, como el drag and drop (utilizado en el editor de niveles), el cual hemos hecho casi un calco de la documentación, adaptando el ejemplo que brinda la misma a nuestro software.

Oportunidades de mejora para la materia y el curso

Si bien hay momentos en los que uno como alumno padece la materia, la realidad es que el curso brinda muchas herramientas para adaptarse a la misma, y es conocido entre los estudiantes y ex-estudiantes de la carrera que es una materia intensa, además de ser dicho por los propios docentes. Es una materia en la cual no te puedes dejar estar y hay que tener mucho tiempo y ponerle mucho empeño para sacarle adelante, pero que el umbral de conocimiento es ampliamente elevado.

Acerca de la materia en sí, creemos que es una materia muy completa y hermosa para cursar, a pesar del sufrimiento. Además, entendemos que el nuevo plan ayuda muchísimo a que los alumnos lleguen mucho mejor a Taller, principalmente por dos cuestiones: la primera, la inclusión de Introducción al Desarrollo de Software, la cual en nuestra opinión personal, es un acierto total de la Comisión Curricular por la cantidad de herramientas que hace conocer al estudiante, que antes de su existencia, desconocía que tan útiles pueden ser para la vida de un desarrollador; y la segunda, el cambio que hubo en la materia Paradigmas de Programación (ex Algoritmos III). Si bien Algoritmos y Estructuras de Datos sigue siendo de las materias más fundamentales de la carrera, Paradigmas hoy quizás le pelea codo a codo por la inclusión de muchos otros temas que antes no se veían, mismo Concurrencia, tema fundamental de Taller!

Acerca del curso no hay mucho que decir, como dijimos hay momentos en los que uno se traba más, pero es una cuestión propia del proceso del desarrollo, y los docentes suelen ser muy atentos a las consultas. Sí creemos que si bien se entiende que en una materia como lo es Taller, no va haber un feedback línea a línea, en los primeros tps estaría bueno que aunque sea haya una especie de esqueleto que el docente asignado llene para decir que features/cuestiones de diseño esperadas por el curso, están y qué no. Esto lo decimos principalmente porque si bien en clase se habla por encima, algunos docentes dieron literalmente cero feedback en los tps individuales. Insistimos, se entiende que no sea algo muy detallado ni mucho menos, pero algo debería haber, aunque sea algo genérico como completar un esqueleto de cuestiones pertinentes al trabajo práctico con SI o NO.

Autocrítica grupal

Creemos que hemos trabajado bien como equipo, siendo buenos compañeros, hablando todos los días sobre que íbamos a desarrollar, cómo lo pensamos encarar, y demás. En cuanto a las problemáticas que tenía algún integrante con alguna parte del tp, buscamos rápidamente reunirnos por llamada de voz en Discord para poder dialogar de qué pensaba cada uno y como lo encararíamos al problema.

Las partes más problemáticas fueron sin duda, el desarrollo de la interfaz gráfica del cliente con SDL y el desarrollo de la lógica del juego. Respecto a QT y la arquitectura en sí, creemos que el conocimiento y material brindado por el curso, junto a la documentación, nos hicieron enfrentar estas tareas casi sin problema alguno.

Como cosas que nos faltaron, como suele ocurrir cuando un programador se entusiasma con el producto, quiere ampliar las herramientas que brinda, y el tiempo no nos sobró cómo para poder hacerlo, o no tanto como nos hubiera gustado. También sin dudas, el código tiene oportunidades de mejora, principalmente desde la legibilidad, porque a nivel complejidad temporal creemos no haber tenido errores *graves*.

El único detalle importante es que en ciertas ocasiones pudimos observar que del lado del cliente hay una aparente Race Condition en el momento del

cierre. No pudimos resolverlo porque solo veíamos un error genérico, y no el error lanzado. Intentamos reproducirlo pero honestamente sucede bastante de vez en cuando, y el único problema que pudimos observar es que la queue del sender se cerrara dos veces, y si bien luego de esto el fallo no volvió a suceder y creemos que quizás era este el problema (o queremos creerlo), no debería ser este el problema porque de ser así el main del cliente debería haber impreso en stderr su correspondiente *exception :: what()*.

Reflexión final para futuros alumnos de Taller

Repitiendo un poco lo que ya dijimos antes, creemos que el estudiante que arranca a cursar Taller con Veiga tiene que saber que va a padecer la materia por momentos, pero realmente se aprende muchísimo, se aprende un lenguaje hermoso como es C++ y abre muchísimo la cabeza acerca de como funcionan los productos de software utilizados en el día a día.

Vengan preparados para esforzarse, pero el camino que se recorre es muy enriquecedor para la vida profesional. Pero cada uno hará su camino en la materia y vivirá su propia experiencia. Caminante no hay camino se hace camino al andar!